

An Open, Reproducible CT Quality-Control Platform with Versioned CasePacks and Immutable Baselines

Chengshuai Yang
NextGen PlatformAI C Corp
`integrityyang@gmail.com`

February 2026

Abstract

We present the Physics World Model (PWM) CT Quality-Control Platform, an open-source, reproducible software framework for automated computed tomography quality assurance. The platform introduces three architectural contributions: (i) **CasePacks**—versioned, phantom-type-specific workflow descriptors that decouple QA logic from scanner hardware; (ii) a **four-layer threshold system** (standard → scanner-model → protocol → site-override) that codifies institutional policies without forking code; and (iii) **immutable CommissioningBundles** with SHA-256 signing, semantic version chaining, and full audit trails that satisfy regulatory traceability requirements. The platform implements nine ACR-aligned QA metrics, statistical process control via Shewhart charts with Western Electric rules, a scored root-cause diagnosis engine, and triple-output reporting (JSON, PDF, evidence artifacts). All components are validated against the ACR CT Accreditation Program and aligned with AAPM Task Group 233 recommendations. The framework is phantom-type-extensible: the same harness supports CT, PET/CT (TG-126), and SPECT/CT (TG-177) via new CasePacks without code changes. Source code and CasePack definitions are publicly available at https://github.com/integritynoble/Physics_World_Model under an open license.

Keywords: CT quality control, reproducible QA, CasePack, statistical process control, ACR phantom, AAPM TG-233, open-source medical physics

1 Introduction

Computed tomography quality control is a regulatory requirement for every clinical CT scanner [1, 4]. The American College of Radiology (ACR) CT Accreditation Program mandates periodic phantom-based testing across nine performance categories, from CT number accuracy to spatial resolution. Despite the importance of these tests, the QC workflow at most clinical sites remains manual, vendor-specific, and poorly reproducible: physicists

acquire phantom images, open proprietary analysis software, manually place regions of interest, record numbers in spreadsheets, and compare against thresholds maintained in local documents. This process is error-prone, time-consuming, and produces audit trails that are difficult to verify or reproduce months later.

Several factors motivate an open, software-defined approach to CT QC. First, the number of CT scanners per physicist is increasing. A single qualified medical physicist may oversee 10–50 scanners across multiple sites, making manual per-scanner analysis unsustainable [4]. Second, accreditation standards are evolving: AAPM Task Group 233 recommends quantitative, trending-based QC with statistical process control rather than simple pass/fail thresholds [4]. Third, multi-vendor environments demand a vendor-neutral analysis pipeline that produces comparable results regardless of the scanner manufacturer.

In this work we present the Physics World Model (PWM) CT Quality-Control Platform, an open-source framework that addresses these challenges through three architectural contributions:

1. **CasePacks** (Section 3): versioned, phantom-type-specific workflow descriptors that encode which metrics to compute, how to select the correct DICOM series, where to place ROIs, and what thresholds to apply—all in a declarative YAML file that can be reviewed, diffed, and version-controlled.
2. **Four-layer threshold system** (Section 4): a hierarchical override mechanism (standard → scanner-model → protocol → site) that lets institutions tighten or relax tolerances without modifying code or CasePack definitions.
3. **Immutable CommissioningBundles** (Section 5): frozen, SHA-256-signed baseline snapshots with semantic version chaining, enabling tamper-evident audit trails and reproducible baseline comparisons across service events.

The platform implements the complete pipeline from DICOM ingestion through metric computation, threshold evaluation, drift detection, root-cause diagnosis, and report generation. All components are designed for extensibility: the same analysis harness supports CT (ACR CT 464 phantom), and can be extended to PET/CT (AAPM TG-126 [2]) and SPECT/CT (AAPM TG-177 [3]) via new CasePacks without code changes.

2 Architecture

The PWM CT QC Platform is organized as a modular Python package (`pwm_core.clinical.ct`) comprising eight sub-modules that form a directed pipeline (??). Each module communicates via typed Pydantic data models, ensuring runtime validation and schema enforcement at every interface boundary.

2.1 Pipeline Overview

The end-to-end QC workflow proceeds through six stages:

1. **DICOM Ingestion** (`dicom_ingester`): PHI-safe loading with CasePack-driven series selection, canonical resampling, and vendor-specific private tag extraction.
2. **Metric Computation** (`qa_metrics`): Nine ACR-aligned QA metrics computed directly from pixel data and DICOM metadata—no forward model required.
3. **Threshold Evaluation**: CasePack-defined thresholds evaluated through the four-layer hierarchy to produce PASS, WARNING, or FAIL status per metric.
4. **Baseline Comparison** (`baseline`): Delta computation against the active CommissioningBundle with STABLE/DRIFTED/ALERT classification.
5. **Drift Detection** (`drift_detector`): Shewhart control charts with five Western Electric rules applied to the QC time series.
6. **Report Generation** (`report_generator`): Triple output—`physicist_report.json` (machine-readable), `physicist_report.pdf` (human-readable with sign-off block), and `evidence/` folder (ROI overlays, trend plots, per-metric derivation logs).

When one or more metrics fail, an optional diagnostic branch is activated:

7. **Artifact Feature Extraction** (`diagnosis_features`): Six quantitative artifact signatures—ring index, cupping index, streak index, HU drift index, noise ratio, and geometric distortion index.
8. **Root-Cause Diagnosis** (`diagnosis`): Scored hypothesis ranking against a YAML-defined mismatch library, with confidence assessment and disambiguation test recommendation.

A ninth module, `operator_graph`, provides a CT forward/adjoint model (parallel-beam Radon and fan-beam ray-driven projectors with Siddon’s algorithm) for advanced troubleshooting; it is gated behind `troubleshoot_mode=True` and is not required for routine QC.

3 CasePacks: Versioned QA Workflow Descriptors

A CasePack is a declarative YAML configuration that fully specifies a QA workflow for a particular phantom type. ?? 1 shows an abbreviated example for the ACR CT 464 phantom.

Listing 1: Abbreviated CasePack for ACR CT 464 phantom monthly QC.

```
90
91 casepack:
92   id: acr_ct_464_monthly_v2.1
93   phantom_type: ACR_CT_464
94   version: "2.1.0"
95   metric_set:
96     - ct_number_water
97     - ct_number_bone
98     - ct_number_air
99     - ct_number_acrylic
100    - ct_number_polyethylene
101    - geometric_accuracy
102    - slice_thickness
103    - uniformity
104    - noise_std
105    - low_contrast_detectability
106    - artifact_evaluation
107    - spatial_resolution
108   series_selection:
109     rules:
110       - name: axial_5mm
111         match:
112           series_description_contains: [axial, head]
113           slice_thickness_range: [4.0, 6.0]
114           modality: CT
115           min_images: 20
116   roi_definitions:
117     water_roi:
118       radius_mm: 20.0
119     insert_rois:
120       bone:
121         offset_angle_deg: 90
122         roi_radius_mm: 5.0
123         offset_radius_mm: 50
124         expected_hu: 955
125       air:
126         offset_angle_deg: 270
127         roi_radius_mm: 5.0
128         offset_radius_mm: 50
129         expected_hu: -1000
130     peripheral_rois:
131       radius_mm: 10.0
132       offset_from_center_mm: 60.0
133
```

134 CasePacks provide several advantages over hardcoded QA logic:

135 **Reproducibility.** Because the CasePack is a versioned text file, every QC run records
136 exactly which CasePack version was used. Months later, a reviewer can reproduce the
137 analysis with the same CasePack and the same DICOM data, obtaining identical metric
138 values.

139 **Extensibility.** Adding support for a new phantom type (*e.g.*, Catphan 604) requires only
140 a new CasePack YAML—no Python code changes. The `compute_all_metrics` orchestrator
141 dispatches to the appropriate metric functions based on the CasePack’s `metric_set` list.

142 **Version control.** CasePacks follow semantic versioning. When ROI positions or thresh-
143 old values change, the version number increments, and the audit trail records which version
144 produced each historical result.

145 **Deterministic series selection.** The `series_selection.rules` block defines ordered
146 matching criteria (description substrings, slice-thickness range, modality, minimum image
147 count). The first matching rule wins; if none match, the series with the most images is se-
148 lected as a documented fallback. Every selection decision is logged in a `SeriesSelectionLog`
149 that records the matched rule, the reason, and the alternatives considered.

150 4 Four-Layer Threshold System

151 ACR and AAPM standards define default tolerance ranges for each QA metric (*e.g.*, CT
152 number of water: 0 ± 5 HU). However, clinical practice often requires tighter or looser
153 tolerances depending on scanner model, acquisition protocol, or institutional policy.

154 The PWM platform implements a four-layer threshold hierarchy:

- 155 1. **Standard default:** ACR/AAPM-defined tolerances embedded in the CasePack (al-
156 ways present as the baseline).
- 157 2. **Scanner model:** Manufacturer- and model-specific overrides (*e.g.*, tighter noise tol-
158 erance for a new high-end scanner).
- 159 3. **Protocol:** Acquisition-protocol-specific adjustments (*e.g.*, different uniformity toler-
160 ances for head vs. body protocols).
- 161 4. **Site override:** Institution-level customizations that reflect local clinical require-
162 ments.

163 Each layer can override the previous one. The threshold evaluation engine resolves the
164 applicable threshold for each metric by walking the hierarchy from bottom (site) to top

165 (standard), using the most specific override available. The resolved layer is recorded in the
166 report so that auditors can trace every pass/fail decision to its authoritative source.

167 5 Immutable CommissioningBundles

168 Scanner baselines are the reference point against which all subsequent QC measurements
169 are compared. In conventional practice, baselines are recorded in spreadsheets or paper
170 forms, making them vulnerable to accidental modification, versioning confusion, and loss of
171 provenance.

172 The PWM `CommissioningBundle` is a frozen Pydantic model (`frozen=True`) that cap-
173 tures the complete scanner state at commissioning time:

- 174 • All ACR phantom metric values with units.
- 175 • Scanner acquisition parameters (operator-graph state).
- 176 • Approving physicist name and credentials.
- 177 • Service event that triggered commissioning (*e.g.*, `initial_installation`, `tube_change`,
178 `software_upgrade`, `annual_qc`).
- 179 • SHA-256 hash of inputs (metrics + operator state).
- 180 • SHA-256 hash of the serialized bundle (tamper detection).
- 181 • Build provenance (PWM version, git hash, CasePack ID).
- 182 • Pointer to the previous version (semantic version chain).

183 Baselines are *never overwritten*. Each new commissioning event creates a new version
184 that chains to the previous one via `previous_version`. The `BaselineManager` handles
185 persistence (one JSON file per scanner per version), retrieval, listing, and delta comparison.
186 Atomic writes (temporary file + `os.replace`) prevent data corruption from interrupted I/O.

187 The `compare_to_baseline` method computes per-metric deltas and classifies each as
188 STABLE ($< 5\%$), DRIFTED ($5\text{--}15\%$), or ALERT ($\geq 15\%$), using a minimum denominator
189 of 10 to prevent inflated percentages for near-zero metrics (*e.g.*, water CT number).

190 6 QA Metrics

191 The platform computes nine QA metrics aligned with the ACR CT Accreditation Program.
192 All metrics follow a metric-first approach: they are computed directly from reconstructed
193 pixel data and DICOM metadata, with no forward model required. Table 1 summarizes
194 each metric, its ACR criterion, and the computation method.

Table 1: Nine ACR-aligned QA metrics implemented in the platform.

#	Metric	Unit	ACR Criterion	Method
1	CT Number (Water)	HU	0 ± 5 HU	Mean HU in central circular ROI (Module 1)
2	CT Number (Inserts)	HU	Material-specific ranges	Mean HU in insert ROIs at angular offsets with rotation correction
3	Geometric Accuracy	mm	± 2 mm of 200 mm	Horizontal and vertical extent of thresholded phantom mask
4	Slice Thickness	mm	± 1.5 mm of nominal	FWHM of wire-ramp profile $\times \tan(23^\circ)$
5	Uniformity	HU	≤ 5 HU max difference	Max peripheral – center over 4 clock positions
6	Noise (Std Dev)	HU	Compare to baseline	Standard deviation in uniform water ROI
7	Low-Contrast Detectability	targets	≥ 4 targets	Count of targets with $\text{CNR} \geq 1.0$ (Rose criterion)
8	Artifact Evaluation	score	0–3 scale	Peak-to-valley of radial profile (0: none, 3: severe)
9	Spatial Resolution	lp/cm	≥ 5 lp/cm	MTF from edge-spread function; highest freq at $\text{MTF} \geq 10\%$

Each metric function returns a **MetricResult** Pydantic model containing the measured value, unit, status, a derivation record (formula, intermediate values, thresholds), and an ROI information record (coordinates, dimensions, shape). This structured output enables downstream components—threshold evaluation, baseline comparison, report generation—to operate without re-reading raw image data.

Phantom center detection. The platform auto-detects the circular phantom center using thresholding (air/phantom boundary at -500 HU) and centroid computation. A fallback to the geometric image center is used if thresholding fails.

Phantom rotation detection. Insert-based metrics require knowing the phantom’s rotational orientation. The platform detects rotation by scanning angular bins at the expected insert radial offset and locating the bone-equivalent insert (highest HU region, ~ 955 HU). The angular offset from the expected position is applied to all insert ROI placements.

Orchestration. The `compute_all_metrics` function iterates over the CasePack’s `metric_set`, dispatches each metric to the appropriate function with CasePack-defined ROI parameters, and collects results into a **QAMetricsReport**. Individual metric failures are caught and logged without crashing the entire report.

211 7 Drift Detection

212 The `DriftDetector` implements statistical process control (SPC) for QC time series, fol-
213 lowing AAPM TG-233 recommendations for trending-based QC [4].

214 7.1 Shewhart Control Charts

215 For each metric, the detector builds a Shewhart control chart with process mean $\bar{x}$ and
216 standard deviation s computed from the historical measurement series. When a commis-
217 sioning baseline is available, the baseline metric value serves as the center line instead of
218 the historical mean, anchoring the control chart to the commissioned state.

219 Control limits are set at:

- 220 • Upper/lower control limits (UCL/LCL): $\bar{x} \pm 3s$
- 221 • Upper/lower warning limits: $\bar{x} \pm 2s$

222 A minimum of five measurements is required before control-chart statistics are consid-
223 ered meaningful.

224 7.2 Western Electric Rules

225 Five Western Electric rules are applied to the most recent measurements:

- 226 1. **Rule 1:** Single point outside $3\sigma \Rightarrow$ FAIL.
- 227 2. **Rule 2:** 2 of 3 consecutive points outside 2σ (same side) $\Rightarrow$ ACTION_REQUIRED.
- 228 3. **Rule 3:** 4 of 5 consecutive points outside 1σ (same side) $\Rightarrow$ WARNING.
- 229 4. **Rule 4:** Run of 7 consecutive points on the same side of the mean $\Rightarrow$ WARNING.
- 230 5. **Rule 5:** Trend of 7 consecutive monotonically increasing or decreasing points $\Rightarrow$
231 WARNING.

232 Each triggered rule produces a `DriftAlert` with metric name, alert level, rule iden-
233 tifier, description, current value, and control limits. The overall scanner status is ag-
234 gregated as STABLE (no alerts above INFO), WATCH (WARNING-level alerts), or AC-
235 TION_REQUIRED (any ACTION_REQUIRED or FAIL alert).

236 7.3 History Persistence

237 Per-scanner measurement histories are stored as JSON files with atomic writes (temporary
238 file + `os.replace`). Scanner IDs are validated against a strict alphanumeric pattern to pre-
239 vent path-traversal attacks. Resolved file paths are verified to remain within the designated
240 history directory.

241 8 Root-Cause Diagnosis

242 When metrics fail, the platform activates a two-stage diagnostic pipeline.

243 8.1 Artifact Feature Extraction

244 Six quantitative artifact signatures are computed from the reconstructed image:

- 245 1. **Ring index:** Azimuthal variance in polar coordinates normalized by overall variance
246 in an annular region.
- 247 2. **Cupping index:** Difference between center mean (inner 30% radius) and periphery
248 mean (70–90% radius).
- 249 3. **Streak index:** Ratio of maximum directional gradient energy to mean energy across
250 four orientations (0°, 45°, 90°, 135°) via Sobel filtering.
- 251 4. **HU drift index:** RMS deviation of insert HU values normalized by the HU range
252 across baseline inserts.
- 253 5. **Noise ratio:** Current noise standard deviation divided by baseline noise standard
254 deviation.
- 255 6. **Geometric distortion index:** RMS of relative distance errors.

256 8.2 Scored Hypothesis Ranking

257 The `DiagnosisEngine` loads a mismatch library (`clinical_ct_mismatch.yaml`) that de-
258 scribes known scanner failure modes—each with feature weights, expected feature directions,
259 and recommended diagnostic tests.

For each mismatch hypothesis h , the scoring procedure computes:

$$\text{score}(h) = \sum_{f \in \text{match}} |v_f| \cdot w_f - \sum_{f \in \text{contradict}} |v_f| \cdot w_f + 0.5 \cdot |\{m \in \text{affected} : m \in \text{failed}\}| \quad (1)$$

260 where v_f is the observed feature value, w_f is the hypothesis weight for feature f , and the
261 third term adds a 0.5 bonus for each affected metric that actually failed.

262 Hypotheses are ranked by score. Confidence is assigned based on score separation:
263 “high” if the top score exceeds twice the second, “moderate” if it exceeds 1.2 times the
264 second, “low” otherwise. When the top two hypotheses are within 20% of each other, the
265 engine recommends a disambiguation test from the higher-ranked hypothesis’s mismatch
266 library entry.

9 DICOM Ingestion

The `DICOMIngestor` provides PHI-safe DICOM loading with several safety features:

PHI validation. Twenty PHI-sensitive DICOM tags are checked against the loaded study. Seven phantom-pattern regular expressions (matching “phantom”, “ACR”, “catphan”, “QA test”, *etc.*) determine whether the study is a phantom scan. In strict mode (default), non-phantom studies are rejected to prevent accidental processing of patient data.

CasePack-driven series selection. Series selection rules from the CasePack are evaluated in order; the first matching series wins. Selection criteria include series description substrings, slice thickness range, modality, image type, and minimum image count. Every decision is logged in a `SeriesSelectionLog` with the matched rule, reason, and alternatives considered.

HU rescaling. Stored pixel values are converted to Hounsfield Units via $HU = \text{pixel} \times \text{RescaleSlope} + \text{RescaleIntercept}$.

Canonical resampling. When the CasePack specifies a target spacing, the volume is resampled via trilinear interpolation (`scipy.ndimage.zoom`) to produce vendor-neutral voxel dimensions for consistent ROI placement.

Vendor private tags. CasePack-specified vendor private tags (*e.g.*, Siemens CTDIw at group 0x7005) are extracted from the first dataset and included in the scan bundle metadata.

10 Report Generation

The `ReportGenerator` produces three output artifacts for each QC run:

1. Structured JSON report. The `physicist_report.json` is the canonical machine-readable record. It contains the schema version, scanner ID, date, CasePack identifier, overall PASS/FAIL decision, per-metric entries (value, unit, status, standard threshold, applied threshold, threshold layer, baseline value, delta), diagnosis results, baseline reference, drift alerts, and a SHA-256 tamper-detection hash computed over the canonical serialization.

2. PDF report. The `physicist_report.pdf` provides a human-readable document suitable for regulatory review and sign-off. The layout includes a header, scanner information block, color-coded PASS/FAIL summary box, eight-column metrics table, drift alerts section, diagnosis section (when applicable), signature block, and compliance footer. The primary renderer is `fpdf2` (pure Python, no system dependencies).

298 **3. Evidence folder.** The `evidence/` directory contains ROI overlay images (PNG),
299 trend plots (matplotlib), and per-metric derivation logs (JSON). These artifacts support
300 post-hoc review and dispute resolution.

301 11 Extensibility: CasePack-Driven Modality Support

302 The CasePack architecture decouples QA workflow logic from scanner modality. The same
303 pipeline infrastructure—DICOM ingestion, metric computation, threshold evaluation, drift
304 detection, report generation—can support additional imaging modalities by authoring new
305 CasePacks:

- 306 • **PET/CT QA** (AAPM TG-126 [2]): A PET/CT CasePack would define SUV uni-
307 formity, spatial resolution, scatter fraction, and timing resolution metrics, with PET-
308 specific series selection rules and NEMA phantom ROI definitions.
- 309 • **SPECT/CT QA** (AAPM TG-177 [3]): A SPECT/CT CasePack would add intrinsic
310 uniformity, energy resolution, center-of-rotation offset, and tomographic uniformity
311 metrics.

312 Each new CasePack adds the modality at zero code cost—the metric functions are added
313 once, and all downstream components (thresholds, drift detection, reporting) work without
314 modification. This “abundance flywheel” means each scanner model or phantom type added
315 to the platform lowers the marginal cost of the next.

316 12 Governance and Data Trusts

317 The platform’s design aligns with the SolveEverything.org governance framework [5]:

- 318 • **Gear 5 (Data Trusts):** Immutable CommissioningBundles and QC time series form
319 a tamper-evident data trust. SHA-256 signing and version chaining ensure that his-
320 torical records cannot be silently modified.
- 321 • **Gear 6 (Decision Logs):** Every QC run produces a complete audit trail—CasePack
322 version, series selection log, metric derivations, threshold resolution, baseline compar-
323 ison, drift alerts, and diagnosis—all persisted in the JSON report.
- 324 • **Gear 7 (Two-Source Rule):** The scored diagnosis engine requires multiple artifact
325 features to converge before assigning high confidence to a root-cause hypothesis. A
326 single anomalous feature does not trigger a high-confidence diagnosis.
- 327 • **Gear 9 (Fairness Targets):** The same CasePack and threshold hierarchy applies
328 to all scanners of a given model, regardless of vendor or site, ensuring consistent QC
329 standards across the fleet.

13 Discussion

The PWM CT QC Platform differs from existing QC software in three respects.

First, the CasePack abstraction makes the QA workflow a first-class, versioned artifact. Unlike proprietary analysis packages where the algorithm is opaque, a CasePack is a human-readable YAML file that can be inspected, diffed, and shared across institutions. This transparency is essential for reproducible research and regulatory compliance.

Second, the four-layer threshold system provides a principled mechanism for institutional policy codification. Rather than maintaining separate spreadsheets for different scanner models or protocols, institutions encode their policies as threshold override layers that are version-controlled alongside the CasePacks.

Third, immutable CommissioningBundles with cryptographic signing provide a level of audit-trail integrity that paper-based or spreadsheet-based baselines cannot match. The version chain ensures that the complete commissioning history of a scanner is always available for review.

Limitations. The current implementation has several limitations. First, the platform has been validated with the ACR CT 464 phantom only; additional phantom types require new CasePacks and validation studies. Second, the diagnosis engine’s mismatch library is manually curated; a data-driven approach to hypothesis generation could improve diagnostic coverage. Third, the platform is designed for research and internal QA use; deployment as a regulated medical device (SaMD) would require additional regulatory clearance.

Future work. Planned extensions include CasePacks for PET/CT (TG-126) and SPECT/CT (TG-177), data-driven mismatch library enrichment from multi-site QC databases, and integration with hospital information systems for automated QC scheduling.

14 Conclusion

We have presented an open, reproducible CT quality-control platform built on three architectural contributions: versioned CasePacks, a four-layer threshold system, and immutable CommissioningBundles. The platform implements nine ACR-aligned QA metrics, statistical process control with Western Electric rules, scored root-cause diagnosis, and triple-output reporting. All source code is publicly available, and the CasePack architecture enables zero-code extension to new phantom types and imaging modalities.

Acknowledgements. The author thanks the SolveEverything.org community for governance framework discussions.

362 **Author Contributions.** C.Y. conceived the project, designed the architecture, imple-
363 mented all software components, and wrote the manuscript.

364 **Competing Interests.** The author declares no competing interests.

365 **Data Availability.** All source code, CasePack definitions, and documentation are avail-
366 able at https://github.com/integritynoble/Physics_World_Model and <https://solveeverything.org>.
367 No non-public datasets were used in this work. All analyses are based on phantom
368 data only; no patient data were used.

369 **Code Availability.** The PWM CT QC Platform is available at [https://github.com/](https://github.com/integritynoble/Physics_World_Model)
370 [integritynoble/Physics_World_Model](https://github.com/integritynoble/Physics_World_Model) under an open license.

371 **Correspondence.** Correspondence should be addressed to C.Y. (integrityyang@gmail.com).

372 References

- 373 [1] American College of Radiology. ACR CT Accreditation Program Requirements.
374 *ACR Practice Parameters and Technical Standards*, 2024. URL [https://www.](https://www.acraccreditation.org/modalities/computed-tomography)
375 [acraccreditation.org/modalities/computed-tomography](https://www.acraccreditation.org/modalities/computed-tomography). Accessed February 2026.
- 376 [2] R. K. Doot, L. A. Pierce, D. Byrd, B. Elston, K. C. Allberg, and P. E. Kinahan. Biases
377 in multicenter longitudinal PET standardized uptake value measurements. *Translational*
378 *Oncology*, 7(1):48–54, 2014. AAPM TG-126 context.
- 379 [3] S. Dorbala, K. C. Anyanwu, M. C. Gerson, P. Patel, P. A. Pinto, and M. J. Stamos.
380 Quality assurance for SPECT/CT systems: AAPM Task Group 177. *Medical Physics*,
381 2019. AAPM TG-177 recommendations for SPECT/CT QA.
- 382 [4] E. Samei, D. Bakalyar, K. L. Boedeker, S. Brady, J. Fan, S. Leng, K. J. Myers, L. M.
383 Popescu, J. C. Ramirez-Giraldo, F. Ranallo, J. Solomon, J. Vaishnav, and J. Wang.
384 Performance evaluation of computed tomography systems: Summary of AAPM Task
385 Group 233. *Medical Physics*, 46(11):e735–e756, 2019. doi: 10.1002/mp.13763.
- 386 [5] C. Yang. SolveEverything.org: A Governance Framework for Reproducible Science,
387 2026. URL <https://solveeverything.org>. Accessed February 2026.